

Research Article

Lite Coding: Core Concepts

Running title: ### Lite Coding ###

Abstract

This article introduces the core principles of a new creative programming framework for time-based artistic practice, *lite coding*. In keeping with the ethos of ubiquitous music, our approach is aimed at supporting everyday (little-c) creativity, knowledge sharing, and computational thinking. The paper documents an online environment for music-making which can be used both for activities in lite coding and to reason about its core concepts (*litePlay.js*). We introduce this JavaScript-based platform, discuss its basic operation and demonstrate key components that enable support for strongly timed execution requirements of browser-based musical activities. Through fully functional online code examples, we describe lite-coding techniques and the *litePlay* programming interfaces. These examples motivate an initial discussion of lite-coding caveats and potentialities and their realisation within browser-based musical scenarios.

Keywords: ubiquitous music computing; lite coding; computational thinking; everyday musical creativity; browser-based programming.

1. Introduction

To deal with emergent aspects of creative coding, we illustrate strategies that feature a close rapport between computational thinking and music-making, highlighting terseness, resilience, scalability, aesthetic pliability, care for the commons and knowledge sharing. We tackle a persistent gap in music-making: How to deal with temporalities based on computational thinking. Our work contributes toward a *ubimus* framework that enables computational thinking (CT) for novices and casual practitioners, *lite coding*. By situating this framework within the context of general creativity studies, we strive to expand creative computing while avoiding aesthetic biases.

This paper builds upon our ongoing work aimed at musical temporalities through a deployment of a web-based lite coding environment, *litePlay.js*.¹ Our proposal is aligned with the ongoing efforts of our community to foster rapid prototyping of ubimus ecosystems (Lazzarini et al. 2020; Yi and Letz 2020). Concomitantly, it inherits from a growing trend in computer science and related fields to foster hands-on, exploratory programming methods (Kelleher and Pausch 2005; Maloney, Resnick, Rusk, Silverman and Eastmond 2010; Ousterhout 1998; Papert 1999; Smith, Cypher and Spohrer 1994; Solomon, Harvey, Kahn, Lieberman, Miller, Minsky, Papert and Silverman 2020; Sorensen and Brown 2007). Through lite-coding examples, we point to alternative forms of temporal organisation highlighting the idiosyncrasies of each implementation as a way to showcase specific choices in design. We provide working-code examples, readily available for usage in sonic-prototyping activities. The remaining sections of the paper include: a description of key features of lite coding; a basic coverage of the design strategies applied in *litePlay.js*; several code examples and detailed analysis of their sonic outcomes and implications for design; and a final discussion situating our findings in the context of the ubimus lite-coding initiatives featuring a preliminary plan for future developments.

1.1 Lite coding

Lite coding is a work-in-progress initiative of the ubimus community (Keller and Lazzarini 2024; Zheng et al. 2023). It involves the materialisation of computational thinking for musical purposes targeting a reduction of complexity to enable learnable and transparent software prototyping.² To tackle the organisation of sonic or multimodal processes, it relies on extant computational languages and browser-based technology.

A tentative target for lite coding is *terseness*, a simplification of means to enable a fast adoption of CT strategies in creative tasks. A typical strategy to achieve terseness involves the usage of abstractions. This view is valid up to a certain point. Consider, for instance, the shorthand labels used in popular music for chord sequences. These labels rely on multiple assumptions that tend to restrict knowledge-construction and sharing. A genre-neutral definition of chords (or

¹ <https://vlazzarini.github.io/litePlay.js/>

² Here we should note that the concept of musical prototype, as introduced by Pimenta et al. (2012), remains valid as a core element of ubimus efforts. It encompasses an expanded notion of creativity mediated by the use of creative surrogates (Keller et al. 2015). And it highlights the iterative dynamics of community-based musical collaboration (McConville 2023).

more accurately, of pitch aggregates) would imply *a perceptually synchronous set of pitched events*. These events do not need to adopt equal temperament: they can employ other tuning systems. Furthermore, they do not need to be restricted to major or minor thirds. Any interval is fair play. The number of stacked events does not need to be limited to 3 or 4. The limit is established by the sonic tool, according to relevant psychoacoustic constraints. Thus, a short-hand label such as C Major represents a very small subset of a large range of available pitch aggregates. Consequently, despite providing some level of abstraction, symbolic chord representations would need to be expanded to encompass alternative tunings or intervallic configurations to become useful ubimus tools. A viable approach to explore terseness involves polymorphism, as it will become clear from the examples below.

Another factor to consider for lite-coding support is *resilience*. As established by ubimus research and related initiatives in other fields,³ resilient platforms demand the existence of engaged communities (Miletto et al. 2011; Pimenta et al. 2012). This not only involves the adoption of open-source libraries, it also entails an explicit adherence to non-commercial, community-oriented ethics. Proprietary technology tends to be brittle. This aspect of lite-coding design is also related to an active defence of the extant digital commons, an attitude that may be labelled *care for the commons*. This respectful approach to the usage and sharing of community-produced knowledge is especially needed in the context of the current expansion of tools that incorporate massive internet-based musical data [cf. Messina et al. 2024].

Another two core qualities of lite coding are *scalability* and *aesthetic pliability*. By emphasising aesthetic pliability, ubimus envisions exploring emergent forms of social interaction which lie beyond the standard division of labour between composers, performers and audience traditionally enforced by the acoustic-instrumental perspective. Several ubimus projects have addressed aspects of pliability by supporting various types of musical materials and by expanding the available strategies for collective decision-making. Complementarily, scalability entails the expansion of the available infrastructure to access larger quantities of resources. This dimension may pose a potential barrier both for the inclusion of geographically distant stakeholders and for handling massive amounts of materials during creative activities. When approached from a ubimus

³ Echoing the ubimus proposals developed since 2007, several researchers are now calling for ‘democratisation’ of resources, more accessibility and inclusivity. This is a healthy change from the elitist emphasis on virtuosity and specialization of the 1990s and 2000s.

perspective, scalability is not just a dimension of efficient infrastructure (as suggested by recent genre-centric proposals geared toward networked music performance). This dimension is also tied to the support of *meaningful* interaction. This issue may be described as the musical counterpart of “the glass cockpit syndrome” (Wiener 1989). Unlimited access to resources, coupled with the unbounded generation of tokens, initially seemed to open the doors to untrained subjects to tackle complex decision-making tasks in musical endeavours. In the context of asynchronous or quasi synchronous musical activities, the observed caveats were not necessarily caused by errors in timing. A recurrent cause of concern was the multiplication of tokens that were not easily understood by all participants. Furthermore, the ability to select relevant computational resources for specific musical aims sometimes may involve fairly advanced cognitive processes to enable classification. Thus, music-making that involves large quantities of resources or procedures tends to produce an explosion of items to choose from. Abstraction strategies are necessary. Nevertheless, to furnish meaningful handles for parametric manipulation and resource selection, the design of creative-action metaphors needs to be tailored for the specific profile of the participants.

A recurring concern of the design ubimus frameworks which is also relevant for supporting lite-coding practices, is *knowledge sharing*. According to Keller and Brown (2017), while acoustic-instrumental designs are aimed toward virtuosic performances, ubimus designs tend to embrace manifestations of everyday creativity. The types of knowledge required by these two approaches to musical interaction are very different. Acoustic-instrumental designs rely on explicit symbolic notation and/or extensive exposure to specialized instrumental training. Ubimus designs rely on local and adaptive, opportunistic usage of social and material resources targeting creative activities that demand fast knowledge acquisition and sharing. The three time-tagging studies discussed by Keller and Brown point to a growing body of evidence indicating that this ubimus creative-action metaphor is well-suited to enable implicit knowledge-transfer in group-based musical activities. The deployments of *jam2jam*, a ubimus system designed both for remote and co-located synchronous musical activities in educational settings, indicate that participants deal with knowledge acquisition and agency at three levels. The first level involves ‘recognition as identification’: participants’ actions are related to the corresponding sonic outcomes. The second targets ‘recognition of oneself’: the ability to separate one’s own sonic proposals from others’. The third aims at ‘mutual recognition’: a process in which the musical exchanges lead to a sense of shared identity. Does lite coding involve similar processes?

Zheng et al. (2023) report on a preliminary deployment of their Emugel lite-coding tool. The results were promising but the most enlightening aspect of their study is the documentation of drawbacks. Zheng et al. classify the difficulties encountered by the stakeholders in two classes: (1) Errors that preclude the successful completion of the task, and (2) Errors that reduce the range of creative outcomes without stopping the subjects from obtaining sonic results. In this study, the first type of errors included Erlang-specific requirements such as the use of commas or difficulties related to handling the prompt-based interface. Some subjects also appeared to have issues interpreting the written information provided in the documentation of the programming environment. The second class of errors highlights the areas in which the lite-coding design could be refined. Analogue-proportional notations were successfully employed for durations and $\times\circ$ notations⁴ proposed for the sequences of percussive events were quickly and effectively incorporated within the subjects' creative cycle. Nevertheless, the envelope values (for dynamics) and the letters and numbers (for pitch sequences) were not readily adopted by the non-musicians that participated in the sessions. These caveats point to two areas that need attention: how to handle time-based parameters and how to deal with symbolic information that relies on domain-specific knowledge. Our *litePlay* implementation targets both gaps and also deals with knowledge-sharing as a multimodal strategy.

Summing up, lite-coding design involves the development of means to enable access to computational thinking within the context of creative musical activities. This framework encompasses terseness, resilience, care for the commons, scalability, aesthetic pliability and a reinforced support for knowledge-sharing. Resilience and care for the commons are interrelated qualities and are grounded on well-established ubimus strategies of open-source and community-oriented efforts toward a sustainable expansion of infrastructure. In particular, our approach builds on top of extant browser-based technology. Terseness and scalability constitute competing forces that indicate a need to develop computational resources through abstractions, without compromising meaningful interaction. This requirement is particularly difficult to achieve when situated in the context of diverse ubimus ecosystems. Aesthetic pliability cannot be sacrificed to accommodate technical issues. Furthermore, knowledge-sharing is a core requirement that needs

⁴ $\times\circ$ notations encompass sequences attacks and silences, represented by the tokens $\times$ and $\circ$ respectively. See an expanded application of the concept in the code examples below.

to address both domain-specific and general creativity factors. Thus, lite-coding tools face multiple pressures that are not necessarily aligned or easy to accommodate.

Having laid out the conceptual scaffold of our proposal for lite coding, we now turn to the technical implications of designing support for creative coding on web-based platforms.⁵ Our hands-on analysis features pointers to working code that can be readily explored on most browsers to replicate the sonic results. We lay out a step-by-step description of core functionalities and point to the potential expansion of the computational environment by means of *JavaScript* and *Csound* functions and opcodes.

2. The *litePlay.js* Environment

The *litePlay.js* library provides a set of *JavaScript* (JS) classes and objects to handle creative music-making. Specifically designed for investigating and reasoning about the concept of lite coding within a wider ubiquitous music context, this programming environment is generic enough to support various approaches and uses. Hence, we can describe *litePlay.js* as a materialization of the lite-coding framework. The library is fairly compact, and is comprised of three elements:

1. *litePlay.js*: the collection of JS code that defines the core of the system as an E6 library.
2. *litePlay.constants.js*: a set of useful constants to support a symbolic approach to coding.
3. *litePlay.csd*, *gm.sf2*: the underlying synthesis and processing code, and sample library, used for music computing. This element employs *Csound* as the audio-processing engine (Vercoe 1985).

2.1 JS Coding Environments

Users can interact with *litePlay.js* directly in a browser by incorporating their code in hypertext markup language (HTML) pages. They can also use a JS read-evaluate-print loop (REPL) interpreter to interact directly with the library. There are various online coding environments available for JS development with different characteristics and support for various kinds of uses. Many of these are directed at web-based application (WAPP) development and can be quite

⁵ The choice of browser-based development, skewing the implementation of a programming language from scratch, is grounded on the advantages of repurposing the extant web-based resources with a particular emphasis on the refined audio-processing functionalities available in *Csound*.

complex, making them unsuitable for lite coding. For the present purposes, we are looking at the following requirements:

1. Allow editing access both to the JS script and the main HTML page code.
2. Provide a means of interaction with the code, so it can be typed and executed interactively
3. Light-weight, with minimal complexity.

Not all online JS coding environments fulfill these. The following three do:

- JS Playground⁶: a lightweight interactive JS code platform with a read-eval-print loop (REPL), allowing editing of HTML and JS (as well as CSS). No support for saving projects at the time of writing.
- Playcode⁷: an interactive JS code platform with the facility for live refreshing/re-running of code, but no REPL. Support for saving projects in user accounts is available.
- p5.js Editor⁸: an interactive JS platform , which is designed for developing p5.js multimedia applications (McCarthy, Reas and Fry 2016), but supports *litePlay.js* very well. It allows editing of HTML and JS code as well as importing other media files into the project. It provides a REPL, and it also supports saving projects in user accounts.

In this paper, we will be employing the facilities provided by the *p5.js* editor environment, through its fully-developed REPL where code can be run interactively. The *p5.js* editor is a web-based application (WAPP) that runs inside a browser. It can be accessed directly from any computing system with an internet connection. Within this arrangement, we can work with *p5.js* programs called *sketches*, which consist of a few additional elements:

1. *sketch.js*: the main JS code that will be run as the program.
2. *style.css*: the basic cascading style sheets for the page display.
3. *index.html*: the web page that loads *sketch.js*, *style.css* and other required scripts.

The sketch as composed of these three elements runs directly on the *p5.js* editor and can be exported as a WAPP. To create a *litePlay.js* application we only require these three files, since the library is picked up (served) from the online code and package repositories. Therefore, a lite-

⁶ <https://www.jsplayground.dev>

⁷ <https://playcode.io>

⁸ <https://editor.p5js.org/>

191 coding system consisting of *litePlay.js* and *p5.js* deployed on a web browser poses no setup
 192 requirements or restrictions to start usage.⁹

193 2.2 A Basic Sketch

194 Now we can show a minimal lite coding WAPP using the elements described earlier. Firstly, we
 195 create a new empty sketch in the *p5.js* editor. This gives the three files that comprise it: we only
 196 need to edit two of them. We add a request to load the `litePlayConstants.js` script to the
 197 `index.html` file. It is possible to see that the HTML page also loads other relevant scripts as
 198 required by the WAPP.

```

199 <!DOCTYPE html>
200 <html lang="en">
201 <head>
202 <script src =
203 "https://cdnjs.cloudflare.com/ajax/libs/p5.js/1.5.0/p5.js">
204 </script>
205 <link rel="stylesheet" type="text/css" href="style.css">
206 <meta charset="utf-8" />
207 <script src =
208 "https://vlazzarini.github.io/litePlay.js/litePlay.constants.js">
209 </script>
210 </head>
211 <body>
212 <main></main>
213 <script src="sketch.js"></script>
214 </body>
215 </html>
216

```

217 The next step is to add code to the `sketch.js` file. When a new sketch is created, the script features
 218 two functions, `setup()` and `draw()`. The first one is invoked when the WAPP is first run. The
 219 second is executed every time a new graphic frame needs to be drawn in the sketch display. For
 220 our needs, we only need to be concerned with `setup()` for the moment. In this function, we will
 221 add code to load up *litePlay.js*, start the synthesis engine, and play a sound:

```

222 async function setup() {
223   // import litePlay
224   const lp = await lpLoad();
225   // play a sound
226   lp.play()
227 }
228

```

9 Please see the documentation of the *litePlay* library for alternative JavaScript editors.

We are using the `lpLoad()` function defined in the `constants` script, which loads the module from where it is served. Another important modification is that we are marking the `setup()` function as `async` so we can use the `await` JS mechanism to load `litePlay` and start the sound engine. This is necessary to order the operations correctly since `litePlay` uses asynchronous scheduling. Note that once we obtain the library, everywhere in the sketch it is referred to through the assigned namespace `lp`. This procedure keeps the `litePlay` symbols separate from other environments (such as `p5.js`), avoiding name clashes¹⁰. Alternatively, if we do not wish to declare an asynchronous `setup()` function we can call `lpRun(code)` to run start `litePlay`, instantiate `lp` as a global variable, and run code asynchronously.

Once the sound engine is running, it is possible to use `litePlay.js` interactively by typing code at the REPL prompt. This is where lite coding meets live coding: the system becomes a fully reactive ubimus computing environment, bypassing much of the complexity found in staple computer music platforms. As we examine the various elements of `litePlay.js`, the code snippets can be directly evaluated at the REPL. When this is not possible, we provide the URLs of the complete sketch.

3. Handling Temporalities

An essential element of any musical activity is its temporal organisation.¹¹ It is therefore crucial that a lite-coding environment enables handling temporalities in a consistent way. To work within the constraints of a *strongly timed* condition, such a system needs to be grounded on a platform that features flexible support of temporalities. JS engines in general have means of delivering the results of computing at predefined times. The main question is whether their performance can be good enough for musical purposes. This question has been raised at different points in the ten or so years since the introduction of audio facilities to Web browsers (Wyse and Subramanian 2013; Wilson 2013). Throughout these years many changes have taken place in JS engines and it is worth our while to reconsider these timing issues at the outset of this study.

10 Sketch URL: <https://editor.p5js.org/vlazzarini/sketches/SRgeiX1cV>

11 It may be argued that a minimal common denominator of any music-making is time. This, of course, goes against the grain of acoustic-instrumental thinking that entails the assignment of agency through actions exerted on an instrument. See Keller et al. (2021) for an initial discussion of ubimus philosophical perspectives.

To evaluate this, we have first put together a prototype environment with only a handful of components, which allowed us to steer the development of the complete *litePlay.js* library with regards to the implementation of its temporal aspects. This minimal system consisted of the basic components of our lite coding environment, running in the *p5.js* online editor. This test environment is comprised initially of a straightforward interface consisting of two functions,

```
instrument.play() // plays a sound immediately
instrument.stop() // stops playing it
```

These instruments are made available in JS as generic objects for emulations of piano, organ, bass, drums, synth and other timbres. It is possible to play them in real time at the *p5.js* console or via MIDI messages by plugging in an external keyboard. With this basic platform we set about developing WAPPs to evaluate different approaches to temporalities within a JS context.

3.1. Using straight JS for Event Timing.

We can initially test the ability of the JS interpreter to handle time. This simple interface is designed for instruments to respond immediately, so we probe the ability of a JS timer to fire events appropriately. We can use a JS timer, such as `setTimeout()`, which allows us to fire an event after a given time. A principle that works extremely well in this environment is recursion, which in this case is specifically called time recursion,

```
cymbals = function (dur) {
  drums.play(cymbal, 0.7);
  setTimeout(cymbals.bind(null, dur), MSEC*secs(dur));
};
```

In this example, we get the drums to play a cymbal sound (with amplitude set to 0.7), and then schedule the function to be run again after a certain duration. To set the duration in beats we have a translating function, `secs()`, which gives the elapsed time in seconds (passed in milliseconds to the JS timer). This way we can set the tempo in BPM. We prime this by calling:

```
cymbals(1);
```

We get a stream of cymbal pulses. We can get functions defined for kicks and snares to set up a rhythm pattern:

```
cymbals(1);
kicks(1.5);
```

286 `snare(2);`

287
 288 Due to the particular way synthesis is implemented, drum sounds do not necessarily need to be
 289 stopped. The repeated event will just replace the previous one. Melodic patterns could use the same
 290 idiom with some modifications to suit specific requirements. For example, by employing two
 291 parameters, `note` (for pitch) and `velocity` (for amplitude), we can apply the conventions of
 292 the MIDI standard. For instance, let's create a bass line:

```
293  basses = function(note,dur,vel=0.6) {
294    bass.play(note,vel);
295    if(vel > 0) vel = 0;
296    else {
297      vel = 0.6;
298      note += 7;
299      if(note > 63) note -= 24;
300    }
301    setTimeout(basses.bind(null,note,dur,vel),MSEC*secs(dur)/2);
302  };
303
```

304 In this case we need to check if a note was actually played and stop it in the next call, using two
 305 timer calls for each sound. We could modify it to use a single call instead, although this is the
 306 absolute simplest way to deal with the issue of starting and stopping sounds. Additionally, we
 307 iterate different pitches by applying the interval of 7 semitones (a fifth), limiting the values to a
 308 given range. Then we can prime the pattern to play one sound per beat, starting from a given note
 309 (51 or D sharp, octave 3, in this case).

310 `basses(51,.5);`

311 This approach seems to work at first. We can set it playing and the expected rhythmic pattern is
 312 obtained as a result. However, after a few seconds, some timing disruptions start to appear in the
 313 bass line, and after a while all other patterns fall off synchronisation. The timing problems start
 314 with the bass pattern because each sound event requires two scheduled timer calls. This mechanism
 315 is prone to delays. Afterwards, the drum pattern is also compromised: the loops get out of sync
 316 with each other. The JS timer simply does not guarantee execution in time. Though it is interesting
 317 that it starts off reasonably well. Sound events based on a single call stay running smoothly for a

little while. Then behaviour gets increasingly irregular. Moreover, any interaction with the browser graphics or screen will delay the timing of the calls further¹².

3.2. Using an Audio Clock

The synthesis engine, based on Csound, is running under the auspices of the Web Audio API, which handles the real-time audio streams coming in and out of JS. Web Audio provides the context for this, encapsulated in the `AudioContext` object. We can access it with a handy asynchronous method from the `Csound` object,

```
audio_context = await csound.getAudioContext()
```

Amongst other things, this gives us the current audio time, which may be neatly packaged as...

```
function audioClock() {
  if(audio_context)
    return audio_context.currentTime;
  else return 0;
}
```

Unfortunately, there is no simple mechanism to fire callbacks using this clock, as it is done with the JS timers. We need to find a solution to apply it to event scheduling. We can start with a simple approach. We can still use the JS timer mechanism to fire events. Having access to a better clock, we can apply a correction to the callback time. This can be done by taking the difference between the current `audioClock()` time and the expected time of the event. When it is zero or greater, it is time to play; otherwise, we do nothing and recurse. This means that events may be scheduled late. But we are able to record how late so we can adjust the time for the next one. The callbacks are supposed to run every millisecond; in practice that is almost never the case. However we can assume that they will be called as soon as possible.

For this idea to work, we need to keep a reference to the time the event was played (old in this code). The update takes into account the time difference at that moment, which indicates how late the event was. The principle is shown in the code below, defining a percussion sequence,

```
percussion = function (instr, vel, dur, old) {
  let t = secs(dur);
  let delta = audioClock() - (t + old);
  if(delta >= 0) {
```

¹² This is demonstrated in the sketch <https://editor.p5js.org/vlazzarini/sketches/gqatdp9V6>

```

349     drums.play(instr, vel)
350     setTimeout(percussion.bind(null, instr, vel, dur,
351         audioClock() - delta));
352 }
353 else setTimeout(percussion.bind(null, instr, vel, dur, old));
354 };
355

```

A similar function is defined for a bass line. As before, it applies an interval to create a 12-tone sequence for pitch movement and splits each sound event into start and stop calls. These sequences are primed with the current audio time to get them running,

```

359 let t = audioClock();
360 percussion(cymbal, 0.8, 1, t);
361 percussion(kick, 0.5, 1.5, t);
362 percussion(snare, 0.5, 2, t);
363 bassline(51, 0.6, 1, t);
364

```

Testing this idea in a sketch, we observe it works well at the start and continues in synchronisation for a longer duration. We can conclude that using the audio clock does make a difference. However, after a couple of minutes the timing pattern is disrupted starting from the bass line. The effect of splitting the sound event in two – start and stop – complicates things. Although it is not really the issue here as the percussion parts also fall apart later. They do so in a different way now: there is a pulse going, which in the previous case was obliterated. This pulse holds together all parts, but the individual events are mistimed, yielding sub-rhythmic patterns. This happens because keeping track of the time errors and correcting for them has some effect but does not address the callbacks that are very late (sometimes beyond 100 ms.). When things fall apart, it appears that for every 4 calls, three are reasonably on time, within a few milliseconds, and one is off by a lot. It is interesting that functions are steadily fired at the start but an irregularity is introduced as time passes. This seems to be related to a loss of precision over the elapsed period because even if we stop all recursive calls and start them again (while the sketch is running), the irregularity does not go away. If we stop running the sketch and re-start, steady timing is restored¹³.

3.3. Using Csound for Time Keeping

The original idea was to have computational instruments with a `play()` action that would make sound immediately, coupled with a `stop()`. This was implemented just by sending two

¹³See the sketch https://editor.p5js.org/vlazzarini/sketches/b_VAISWsR

MIDI messages to Csound, NOTEON and NOTEOFF. However, after the first experiments, we concluded there is no simple way to schedule these accurately. Therefore we now change our approach. The idea is to add a when parameter so the sound can be scheduled to happen at some precise time in the future (or now, if the time is zero),

```
instr.play(when, ...)
instr.stop(when, ...)
```

The Csound scheduler then acts diligently to count time without any further delay (beyond the one involved in the JS call which is hopefully negligible). The beauty of this is that now we can work independently of the JS timer: we can compute the required time for the next event and just schedule it ahead. The `setTimeout()` function is still used, exclusively as a means to allow the recursion to work without locking the JS interpreter. We ignore its exact timing; as long as it gets called repeatedly without an exceptionally big break, we are good. We can adopt the previous strategy to use the audio stream clock as a reference. All we need to do is to slot in the new `play()` call and give it the corrected time,

```
percussion = function (instr, amp, dur, old) {
  let t = secs(dur);
  let delta = audioClock() - (t + old);
  if (delta >= 0) {
    drums.play(t - delta, instr, amp);
    setTimeout(percussion.bind(null, instr, amp, dur,
      audioClock() - delta));
  } else setTimeout(percussion.bind(null, instr, amp, dur, old));
};
```

In the case of the bass line, we can also schedule when to stop, which solves part of the difficulties we had before,

```
bassline = function (note, amp, dur, old) {
  let t = secs(dur);
  let delta = audioClock() - (t + old);
  if (delta >= 0) {
    bass.play(t - delta, note, amp);
    bass.stop(t - delta + t * 0.9, note);
    note += 7;
    if (note > 63) note -= 24;
    setTimeout(bassline.bind(null, note, amp, dur,
      audioClock() - delta));
  } else setTimeout(bassline.bind(null, note, amp, dur, old));
};
```

Finally, we seem to have achieved a result that supports strongly timed requirements, as demonstrated by this new sketch¹⁴. A combination of the audio clock for reference and the Csound scheduler gives us a means of delivering a steady stream of events. Through code analysis, we were able to prove that this mechanism is very resilient¹⁵, yielding stable outcomes for periods of unspecified length. It is also possible to stop the sequences by redefining the callbacks to stop recursion, starting again without any issues. With this particular aspect secured, we are able then to concentrate on supporting lite coding with a feature-full interface.

4. Lite Coding Interfaces

The *litePlay.js* environment provides a set of interfaces to support lite coding. With these tools we can investigate and reason about the operational aspects of this ubimus activity. A basic interface in the environment is provided by a `play()` action that may be invoked on different objects. Depending on what these are, slightly different interpretations of play are deployed. As we have shown earlier, we can start at the outer scope of the platform and say:

```
lp.play()
```

to simply play a sound until stopped. We may assign a specific instrument and similarly use:

```
lp.organ.play();
```

being more particular about the sound we are asking for. Also, when an event starts, it stays on until we explicitly stop it.

```
lp.organ.stop()
```

and so on, the interface also includes more complex musical objects that can be employed in different ways, as we will see later.

4.1 Events

¹⁴See <https://editor.p5js.org/vlazzarini/sketches/ifsOSPL7r>

¹⁵ While the full proof is beyond the scope of this paper, it is obtained through the deterministic time guarantees provided by *Csound* running on an Audio Worklet. Any deviation from accurate timing will also lead to audio drop outs (in which case we have a breakdown of the clean audio constraint that is assumed to exist before any strongly timed computing conditions can be measured).

The *litePlay.js* environment employs events to represent the sounds that happen when certain computing actions occur. These may have different characteristics which are defined with plain terms.

4.1.1 What?

To start with, *what* shall we play? The system should be able to respond in ways that are recognisable by the user. If we are considering, for instance, common-practice instrumental music, then *what* could be for instance, a tone, defined in scientific pitch notation, or perhaps an unpitched sound from a standard collection of samples. Now we can say, for instance:

```
lp.play(Eb5)
```

and we get a sound with a pitch corresponding to E-flat in the 5th octave (with a default A4 standard tuning of 440 Hz). This symbolic approach has also a numeric counterpart, so that a pitch can be defined in a linear semitone scale, based on the MIDI note range, from 0 to 127, where 60 is equivalent to middle-C (~ 260 Hz):

```
lp.play(60)
```

Note that while we use this scale as a straightforward way to express the logarithmic nature of pitch, values are not limited to 12-tone equal-tempered tuning. We can use fractional parts to denote any microtonal deviations, such as 60.01 defining a 1-cent sharp middle-C. We can also implement this idea through a symbolic approach,

```
lp.play(C5+quarterTone)
```

to get in this case a quarter-tone upward deviation.

Furthermore, the adoption of symbols or tokens allows us to refer to unpitched sounds. For example, we can also say

```
lp.drums.play(snare)
```

to get a snare drum sound. The instrument interface supports multiple sounds from a single instrument generated in synchrony,

```
lp.organ.play(C4, E4, G4)
```

and thus we hear a major chord. Equally, if we want a sequence of pitches, the generic `play()` command should respond appropriately,

```
lp.play(C4, E4, G4)
```

giving us an arpeggio. Note the polymorphism here: by asking more than one sound from a computational instrument, we may get simultaneous events (held until we issue the stop-playing request); conversely, when we ask the system to play something, *litePlay.js* gives us the sounds in sequence (all lasting the same durations). Furthermore, a generic attribute can be made to fit whatever specification we want, just by complementing the other parameters.

4.1.2 How Loud?

Now we may want to go one step further and give specific characteristics to the sounds we play. So far, all sounds played on a given instrument have the same loudness level. But the interface should allow us to set the intensity of each sound. We need to find a way to provide both *what* and *howLoud* for each sound played,

```
lp.play([C4, 0.1], [E4, 0.5], [G4, 0.9])
```

We can now group these two attributes together using square braces `[.]`, which is facilitated through the use of JS arrays. The resulting arpeggio is then played *crescendo*.

4.1.3 When?

Each one of the arpeggio tones above is separated from the previous event by the same amount of time (which we may call one *beat*). What if we want to disrupt this pattern and make the second sound come earlier in time? As before, the interface should allow us to do this, and we may use the same method to group the attributes in a simple way, adding the *when* to it,

```
lp.play([C4, 0.1, 0], [E4, 0.2, 0.5], [G4, 0.4, 2])
```

This results in a first sound triggered immediately (zero delay), a second one half a beat later, and a third two beats after the first.

4.1.4 How Long?

We have already noted that in the case of the generic `play()` interface each sound has the same duration. Changing its start time (*when*) does not make this any different. So we can add another attribute to define how long it should last,

```
lp.play([C4, 0.1, 0, 3], [E4, 0.2, 0.5, 0.5], [G4, 0.4, 2, 0.1])
```

In this case, the first sound will be held for three beats, the second one for half a beat, and the third for one tenth of a beat.

4.1.5 On Something?

Given that different instruments can be used, it would be good to control this from the generic `play()` interface, so that we can ask which instrument object would be activated. So far, we have only heard a `piano` sound, which is what we get when the system is started. We can modify this by defining the global `instrument()` through this statement,

```
lp.instrument(lp.synth);
```

which sets output to a given timbre (or instrument object, in computational terms). It can now be played with the usual *what* parameters,

```
lp.play(F5, D6);
```

This is a first step, but it may be limiting. The interface should also allow us to give each sound a specific parameter to render a targeted timbre:

```
lp.play([C4, 0.3, 0, 3, lp.organ],
        [E4, 0.6, 0.5, 0.5, lp.synth],
        [G4, 0.9, 2, 0.2, lp.piano]);
```

Thus, the first C4 event yields an organ sound. The second E4 event is played by the `synth` instrument. And the third G4 sound is played by an emulated `piano`. The *onSomething* concept is fulfilled by a large set of instrument objects provided by *litePlay.js*.

4.1.6 Event Objects

These concepts can be collected into a single component of the *litePlay.js* programming interface, the `event` object. This is effectively a plain JS array,

```
[what, howLoud, when, howLong, onSomething]
```

defining the basic attributes of a lite-coding event. Note that any missing parameters are replaced by default values, allowing its abbreviated usage. At its simplest, we can dispense with the array form and just pass a *what* parameter, as we have shown in the first example, but complete events can have the full set of five arguments. Events can also be handled through the instrument-object `play()` interface. In this case the final parameter is a non-op (given that the *onSomething* to be played is fixed),

```
lp.organ.play([C4, 0.1, 0, 3],
              [E4, 0.2, 0.5, 0.5],
              [G4, 0.4, 2, 0.1]);
```

4.2 Event Lists

By grouping events together, we can construct `eventList` objects, which can contain an entire musical structure, akin to the acoustic-instrumental notion of *a score*. An `eventList` can be anonymous:¹⁶

```
lp.eventList.play(0, [[C4, 0.1, 0, 3],
                     [E4, 0.2, 0.5, 0.5],
                     [G4, 0.4, 2, 0.1]]);
```

or it can be held in a variable for re-use and further manipulation,

```
let evt = lp.eventList.create([C4, 0.1, 0, 3],
                             [E4, 0.2, 0.5, 0.5],
                             [G4, 0.4, 2, 0.1]);
evt.play();
evt.add([C3, 0.1, 3, 3]);
evt.play(3);
```

The first (optional) argument to `play()` is a time offset, followed by a list of events (an array of event objects). Means of event insertion, removal, and list clearing, and to obtain the data as a score object (containing the list of events and a `play()` method to act on it) are available in the `eventList` interface. Figure 1 summarises all the various forms of the `play()` interface and its relationships to *events*, *eventLists*, and *instruments*.

¹⁶ Check a general JS programming reference for details on object declarations.

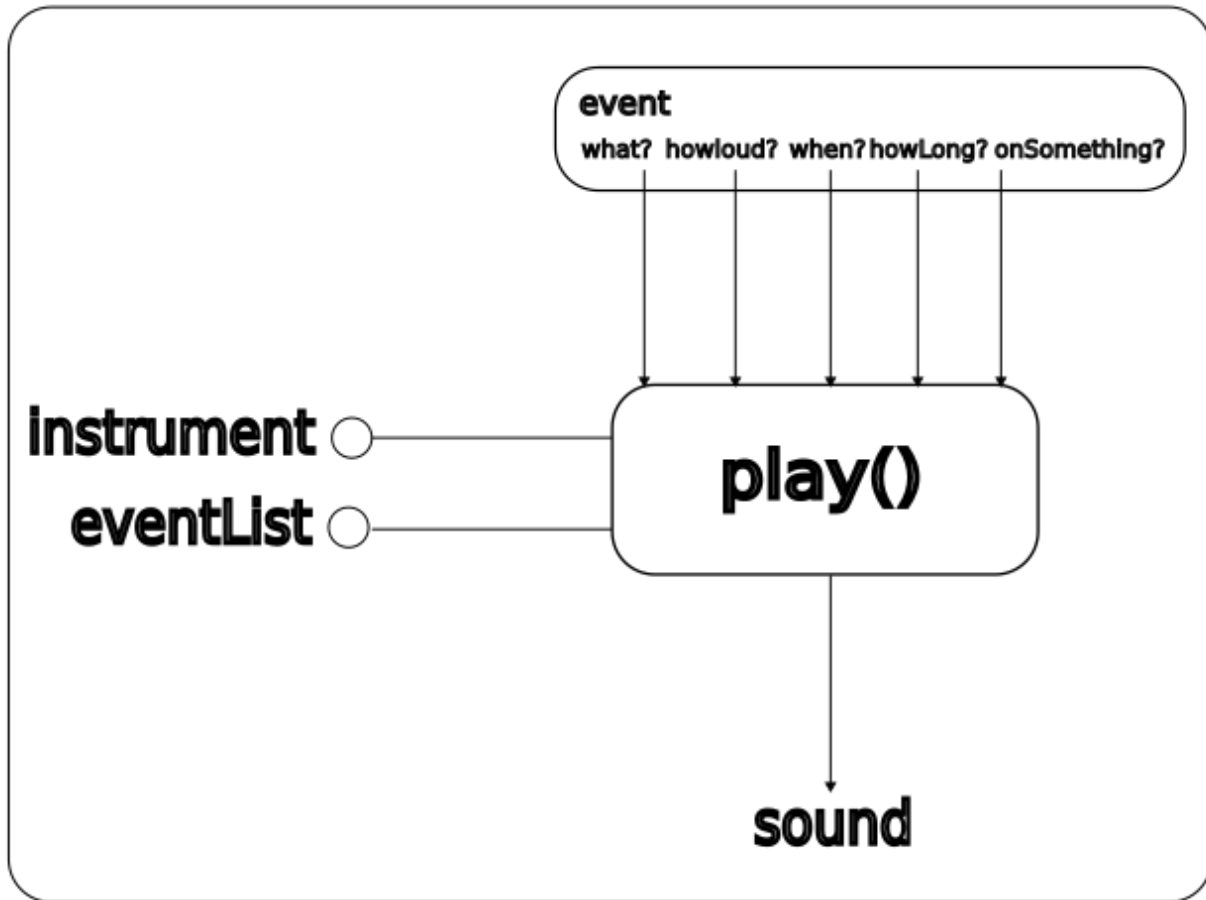


Figure 1. The `play()` interface.

4.3 The Polyphonic Sequencer

The various polymorphisms of `play()` are all strongly timed as discussed earlier. However, each call will be started within an undefined time interval from invocation. That means that either when the script containing `play()` is run or when we are executing it interactively from the REPL prompt, there will be a non-zero delay which will be applied to the event or events being triggered. In a single call, all events will have a definite and unchanging time relationship, but two calls in a sequence cannot be guaranteed to be separated by a specific time interval.

While the `eventList` interface allows a time offset to be provided, this is also dependent on the asynchronous execution of the JS calls that do not have a strongly timed characteristic (which is provided by *Csound* for all events in a list). For this condition to be achieved more generally,

we need to impose a *time grid* on which we can hang our events or event lists. Within this scenario, we can provide means to trigger various *play* actions in strict time relationships with each other.

Furthermore, we recognise that one of the most typical approaches to music-making involves the use of repeated patterns, and that users may expect this to be provided in a lite-coding environment. This is supported by the *polyphonic sequencer* interface. It handles the creation of objects that maintain a strict time count. It is then possible to set up sequences to be played repeatedly.

Unsurprisingly, event information is supplied to sequencers in the form of event lists: a concise, yet flexible and logical representation of polyphonic musical sequences. Any number of sequences (up to the computation limits of the hardware) can be set up to play in strict synchronisation. Let's see an example of a drum kit beat pattern consisting of three instruments. We first lay out the sequences,

```
shuf = [[cymbal, 1, 0, 1 / 3], 0, [cymbal, 0.9], [cymbal, 0.9]];
kck = [[kick, 0.5], 0, [kick, 1]];
snr = [snare, 0];
```

The event lists are organised in beats, each element of the outer array is to be played at a separate tick. The timing in beats per minute (bpm) is defined by a global setting (`lp.setBpm()`), featuring an optional beat division. A silence falling on a beat is represented symbolically by an 0.¹⁷ So a simple sequence, `snr`, is a snare hit (with all other default params) followed by an empty beat. For the `kck` sequence we have events with sound (*what*) and amplitude (*howLoud*) parameters. For the `shuf` sequence, we have a first event with a given duration (*howLong*) as well. If a time (*when*) is used in the event list, it is interpreted as an offset to the nearest time grid tick. Note that each sequence has a different length in beats. This allows the creation of interlocking polyrhythms very easily.

Once the sequences are set, we can create a sequencer object to insert them,

```
const beatDiv = 1 / 3;
sequencer = lp.sequencer;
cymbals = sequencer.add(drums, shuf, 0.5, beatDiv);
kicks = sequencer.add(drums, kck, 0.1, beatDiv);
snare = sequencer.add(drums, snr, 0.1);
```

¹⁷ For a complementary implementation example, see Zheng et al.'s `xo` notation.

599

600 The first argument to the `add()` method is the default instrument being used (which can
 601 be overridden by a new assignment). Then we have the event list, followed by a global amplitude
 602 offset (`0.5`). The final optional argument is a beat division, which makes the sequencer interpret
 603 each beat in the event list according to a quantised time grid ($\frac{1}{3}$ of a beat in this case). The sequence
 604 can be played faster (division < 1) or slower (division > 1) in relation to the set BPM. In this case,
 605 each beat of the `cymbals` and `kicks` sequence is $\frac{1}{3}$ of the length of the beat played by the `snare`s.
 606 A certain polyrhythm ensues, creating an interesting shuffled beat pattern. This sequencer can be
 607 prompted by the `play()` method of the sequencer object,

608 `sequencer.play();`

609 The sequencer is sophisticated enough to allow for complex timed structures consistent
 610 with standard metric organisation. We can place any number of events on a single beat by
 611 employing an event list with more than one event. For example, each time point can consist of
 612 several complete events (including individual *onSomething* parameters), incomplete events
 613 containing only some parameters, just *what* indications, or silence, as shown here:

```
614 melody = [
615     [[Eb5, 3, 0, 2, organ], [Bb5, 3, 0, 2, organ], [G6, 1, 0, 2]],
616     [[F5, 3, 0, 2, organ], [Ab5, 3, 0, 2, organ], [Db6, 1, 0, 3.5]],
617     O,
618     [Bb5, 1, 0.5, 0.5],
619     [[Eb5, 3, 0, 1, synth], [G5, 3, 0, 1, synth], [C6]],
620     O,
621     Bb5,
622     Ab5,
623 ];
```

624

625 A summary of the sequencer interface is shown in Fig. 2.¹⁸

18 For a full example see <https://editor.p5js.org/vlazzarini/sketches/c4PhzF39r>

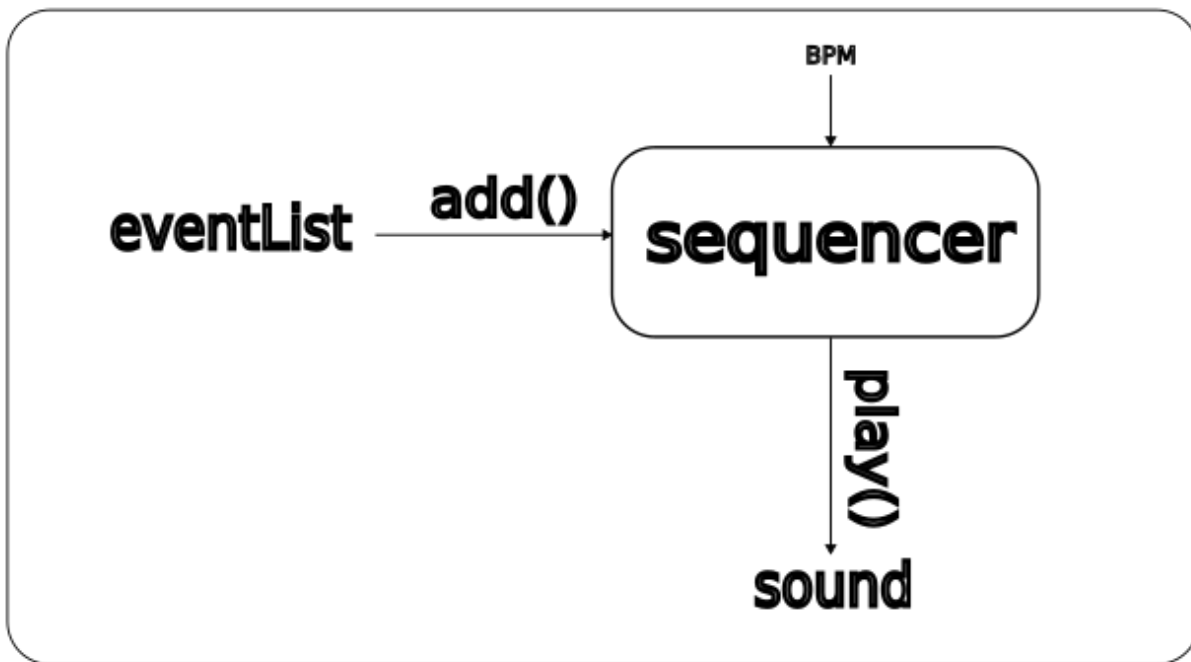


Figure 2. The sequencer interface.

4.5 Generators

Each event parameter in *litePlay* (with the exception of *onSomething*) can be made fully dynamic. In addition to static values (symbolic or numeric), we can also pass function objects in events for *what*, *howLoud*, *when*, and *howLong*. We call these *generators*: functions that take no arguments and return a value that may be suitable for the given parameter. For example,

```

let things = [F4+tenCent, F4+quarterTone, F4+thirdTone,
              F4-tenCent];
let what = ()=> {
  return things[int(random()*things.length)]
};

```

which gives us a random selection from an array of *things*. We can use it in an event list, for example,

```

seq = [[what, 0.2, 0, 0.75], [what, 0.2, 0, 0.75]]

```

and we will have a sequence of random deviations from an F4.

The *litePlay.js* environment provides a set of generators for random values for each event parameter in three bands (*low/short/now*, *mid/soon*, *high/long/later*). For example,

```
seq = [[lp.lowPitch, lp.midLevel, lp.soon, lp.longDuration], 0];
```

if sent to a sequencer it will produce a sequence of random pitches, amplitudes, start times and durations. That sequence of course will be played in a strictly-timed way, but because of the time offsets, events will not likely fall on the beats. Note that each iteration the generators will be producing new values for each event, rather than repeating statically computed data. This is a sophisticated mechanism that can be used to add non-deterministic elements to lite coding in a simple and flexible way, although we recognise that it brings some level of conceptual complexity to ubimus computing¹⁹.

4. Discussion

Recapitulating the key characteristics of our approach to lite coding, we have presented a browser-based programming environment designed to support computational thinking in music-making without demanding intensive training or specialized knowledge. The *litePlay* library features support for the transparent construction of events, featuring five parameters in a JS array: `[what, howLoud, when, howLong, onSomething]`. `what` refers to both pitched and unpitched sounds, without restrictions of tuning systems. `howLoud` stands for parametric control of intensity. `when` is employed to define the onset, encompassing either synchronous or delayed values. `howLong` provides parametric control of duration. And the slot `onSomething` is employed for the timbral characteristics of the event.

Events can be concatenated in sequences by means of event lists. Event lists provide placeholders for metric-based temporal organizations, featuring support for handling time grids (based on a beat or its subdivisions). Given that each sequence can have its own temporal organisation, and provided a sequencing mechanism to render multiple parallel sequences, complex meter-based systems can be deployed. The use of subdivisions of the beat enables the typical unified meter found in common-music practice notation and it also allows polyrhythmic and additive rhythmic processes featured in many musical trends of the 20th century.

19 Generator example sketches: <https://editor.p5js.org/vlazzarini/sketches/-bjOcWkYP>, <https://editor.p5js.org/vlazzarini/sketches/4TmauZabd> and https://editor.p5js.org/vlazzarini/sketches/fQwLCU_e4

Furthermore, a generative mechanism enables turning four of the five event parameters into dynamic processes. This feature simultaneously simplifies the syntax and enhances the temporal strategies available. For instance, rather than having a fixed *howLong* value, three ranges within random distributions can be defined through the means of *generators*: `lp.shortDuration`, `lp.midDuration`, and `lp.longDuration`. When iterated through a sequence, the `lp.shortDuration` generator will yield mostly short events, `lp.longDuration` will render long events and `lp.midDuration` will furnish average durations. Taking into account that each iteration always produces new values, the chances of repetitions are very low. This means that a syntax encompassing three tokens yields a large number of parametric variations. This time-organisation mechanism complements the deterministic metric-based approach and provides opportunities to explore hybrid temporalities.

How does litePlay contribute to the design aims of lite coding? We have introduced a strategy to achieve *terseness*, polymorphism. Through this technique, lite-coding events can range from a minimal *what* parameter to fully determined sonic phenomena. Granted, we have not addressed dynamic changes of timbre. This is among the targets of the next design cycle of litePlay.

We have also started to explore paths to *scalability*. As previously stated, this design quality is not limited to the ability to expand the resources, it also demands *meaningful* interaction. We have implemented a simplified syntax, encompassing three tokens that yield varied outcomes applicable to four sonic parameters. This provides a richer palette of creative processes without increasing the investment in parametric input. Furthermore, there is thorough provision for meter-based deterministic temporalities. Hence, aesthetic pliability is supported.

Our strategy to ensure resilience and care for the commons involves relying on two community-oriented open-source resources, *Csound* and *p5.js*. Arguably, we have only explored a fraction of the functionalities supported by these tools. For lite-coding objectives this is a feature, not a caveat. We are interested in a deployment-driven design cycle. This method can be complemented with speculative explorations involving concepts that are not yet fully shaped to be field-tested or by musicology-based assessments entailing the use of the extant musical repertoire to inform design decisions (cf. Keller and Ferneyhough 2004). Nevertheless, both resilience and care for the commons also depend on a parsimonious approach to the deployment of new technology.

Finally, knowledge-sharing has been partially supported by adopting a semantic-oriented approach to the programming-interface design. We have employed tokens for parameters compatible with approximate values, *short*, *mid*, *long* or *low*, *mid*, *high*. We have also made sure most parameters were supported by both number-based and symbolic input. Hence, we pave the way for the incorporation of automated text-handling techniques.

5. Conclusions

In this article, we introduced the concept of lite coding and demonstrated its potential through the development of a web-based environment consisting of a JavaScript library, *litePlay.js* deployed by means of the *p5.js* interactive editor. We discussed several key aims of lite coding: enabling learning and knowledge-sharing through ubiquitous music activities, leveraging the power of computational thinking to remove barriers to novices and fostering flexible approaches to temporalities to promote a diverse ecology of ubimus practices. To tackle creative musical activities within a strongly timed coding environment, we investigated alternative approaches to handling temporalities using browser-based technology. Through a detailed description of our design process, the featured examples indicate that it is possible to achieve accurate scheduling within a JS platform by taking advantage of the solid support for timed execution provided by *Csound*. Furthermore, our results indicated that alternative strategies to temporal organisation are possible if the myth of delay and jitter as an insurmountable limitation is discarded.

We also documented the basic interfaces provided by *litePlay.js*. One of the motivations for the development of such an environment is to support the process of reasoning about lite-coding techniques and limitations, highlighted throughout this paper. Through an incremental approach to design, we managed to share first principles (such as the *play* action or the concept of events featuring five conceptual parameters), and two contrasting strategies to handle temporalities. To complement this introduction to *litePlay.js*, a startup tutorial for the system is also available online²⁰.

We acknowledge the need for field assessments of the proposed techniques, since our focus group only features trained musicians. Musical activities may involve fast engagements in basic tasks (i. e., generating and modifying one sound), or collective structured tasks (such as iterated

²⁰ <https://vlazzarini.github.io/litePlay.js/tutorial.html>

sequential composition), or they could target open-ended endeavours in which lite coding serves as a prototyping platform within varied toolsets. In any case, we envision a development closely tied to field studies that provide opportunities for the expansion of everyday creativity.

References

1. Kelleher, C. & Pausch, R. (2005). Lowering the barriers to programming: A taxonomy of programming environments and languages for novice programmers. *ACM Computing Surveys* **37**(2), 83–137.
2. Keller, D. & Brown, A. R. (2017). Knowledge transfer in ubiquitous musical activities. In *Proceedings of the Brazilian Symposium on Music Research (SEMPEM 2017)*. Goiânia, GO: UFG.
3. Keller, D. & Ferneyhough, B. (2004), Analysis by Modeling: Xenakis's ST/10-1 080262. *Journal of New Music Research* **33**(2), 161–171.
4. Keller, D. & Lazzarini, V. (2024). Building blocks for lite coding: Temporalities in litePlay.js. In *Proceedings of the Ubiquitous Music Symposium (UbiMus 2024)*. Macau, China: Ubiquitous Music Group.
5. Keller, D., Miletto, E. M. & Otero, N. (2015). Creative surrogates: Supporting decision-making in ubiquitous musical activities. In *Proceedings of the 3rd International Conference on Computation, Communication, Aesthetics and X (xCoAx 2015)* (pp. 168–183). Glasgow, Scotland: xCoAx.
6. Lazzarini, V., Keller, D., Otero, N. & Turchet, L. (eds.) (2020). *Ubiquitous Music Ecologies*, London: Taylor & Francis (Routledge).
7. Maloney, J., Resnick, M., Rusk, N., Silverman, B. & Eastmond, E. (2010). The Scratch Programming Language and Environment, *ACM Transactions on Computing Education* **10**(4), 1–15.
8. McCarthy, L., Reas, C. & Fry, B. (2016). *Making Interactive Graphics in JavaScript and Processing*, San Francisco, CA: Maker Media.
9. McConville, T. (2023). The Artist as a Subscription: Patching music as an artistic device. *Organised Sound* **28**(3), 352–361.
10. Messina, M., Keller, D., Freitas, B., Simurra, I., Gómez, C. & Aliel, L. (2024). Disruptions, technologically convergent factors and creative activities: defining and delineating musical stuff. *Digital Creativity* **35**(1), 13–30.
11. Miletto, E. M., Pimenta, M. S., Bouchet, F., Sansonnet, J.-P. & Keller, D. (2011). Principles for music creation by novices in networked music environments. *Journal of New Music Research* **40**(3), 205–216.
12. Ousterhout, J. K. (1998),. Scripting: higher level programming for the 21st Century. *Computer* **31**(3), 23–30.
13. Papert, S. (1999), *Logo Philosophy and Implementation*. Denver, CO: Logo Computer Systems Inc.
14. Pimenta, M. S., Miletto, E. M., Keller, D. & Flores, L. V. (2012). Technological support for online communities focusing on music creation: Adopting collaboration, flexibility and multiculturalism from Brazilian creativity styles. In Naheb Amin Azab (ed.), *Cases on Web 2.0 in Developing Countries: Studies on Implementation, Application and Use*. Vancouver, BC: IGI Global Press.

15. Smith, D. C., Cypher, A. and Spohrer, J. (1994). KidSim: programming agents without a programming language. *Communications of the ACM* **37**(7), 54–67.
16. Solomon, C., Harvey, B., Kahn, K., Lieberman, H., Miller, M. L., Minsky, M., Papert, A. & Silverman, B. (2020). History of Logo. *Proceedings of the ACM on Programming Languages* **4**(HOPL), 1–66.
17. Sorensen, A. & Brown, A. R. (2007). aa-cell in practice: an approach to musical live coding. *Proceedings of the International Computer Music Conference* (pp. 292-299). Copenhagen. ICMA.
18. Vercoe, B. (1985). *Reference Manual for the Csound Audio Processing System*. Cambridge, MA: MIT Media Lab.
19. Wiener, E. L. (1989). *Human Factors of Advanced Technology* (“Glass Cockpit”) *Transport Aircraft*. https://human-factors.arc.nasa.gov/publications/HF_AdvTech_Aircraft.pdf
20. Wilson, C (2013). *A Tale of Two Clocks*, <https://web.dev/articles/audio-scheduling>, accessed 21.03.25.
21. Wyse, L and Subramanian S. (2013). The Viability of the Web Browser as a Computer Music Platform, *Computer Music Journal*, **37**(4) (Winter 2013), 10-23.
22. Yi, S. & Letz, S. (2020). The browser as a platform for ubiquitous music. In V. Lazzarini, D. Keller, N. Otero & L. Turchet (eds.), *Ubiquitous Music Ecologies* (pp. 185–206). London: Routledge.
23. Zheng, R., Keller, D., Timoney, J., Lazzarini, V. & Messina, M. (2023). Toward lite coding: Designing the Emugel prototype to boost music-computational thinking. In *Proceedings of the Ubiquitous Music Symposium (UbiMus 2023)*. Derry, Northern Ireland: Ubiquitous Music Group.